

无线电干扰源的快速自动定位与清除模型

摘要

【待填充：开头总述，一句话交代背景与要解决的问题，一句话说明本文总体上采用什么模型与算法体系，并串起四问“单次定位 → 检测点布置 → 多源清除 → 定向源辨识”。】

针对问题一，基于示向度误差有界的特点，本文建立了有界区间角楔交会模型，把每个检测点的示向度按误差界张成角楔，干扰源必在各角楔交会的公共部分中。由于形成的定位区域为凸多边形，直径归结为顶点集上的最远点对。使用程序模拟构造的五组算例直径在 10.59 ~ 25.64 m 之间，而以该直径为直径的圆并不总能覆盖定位区域，五组算例中有两组失败，越界顶点到圆心距离超过半径约 10.6%。

针对问题二，只掌握一个检测点处的示向度、源点距离未知，无法先定位再布点，故建立以期望定位区域直径为准则的布点模型：由两探测束的交会几何写出定位区域直径，并对源点位置与第二次测向的误差求取期望，得到只依赖第二个检测点相对第一个检测点位置 (a, b) 的二维价值函数 $E[D](a, b)$ ，使用理论方法求解与数值模拟方法求解获得结果吻合。我们选择一个合理阈值 60m，则第二检测点和候选区域即为相对位置满足 $E[D] < 60m$ 的区域 $C = \{(a, b) | E[D](a, b) < 60\}$ ，具体区域形状见下正文。

针对问题三，【待填充：多源定位与清除策略及实测指标。】

针对问题四，【待填充：含定向源的检测与清除策略及实测结果。】

【待填充：结尾总评，四个模型的优点、缺点各一两条，并给出可继续改进的方向。】

关键词：示向度交会 直径圆覆盖

一、问题背景与重述

1.1 问题背景

随着无线电技术的普及和广泛应用，频谱管理中的干扰源定位与清除问题日益突出。干扰不仅影响正常通信，还可能危及公共安全。传统清除方式通常受环境遮挡、低功率发射等影响，精准定位耗时长，且危险环境下人工作业风险高，因此研发机器狗自动定位清除系统代替人工执行任务具有现实需求。但任务中通常面临着目标区域范围较大，干扰源数量、频道、位置、有效接收半径及类型未知，测向存在误差，移动、检测、切换频道和清除过程存在耗时，定向干扰源的发射方向无法辨识等困难。如何在信息不完全且时间受限的条件下高效完成任务，是该自动定位清除系统亟待解决的问题。

1.2 问题重述

考虑到上述实际情况中面临的困难，本文需依次解决以下四个问题：

1. 已知若干检测点的位置及干扰源关于这些检测点的示向度，给出用交会定位法计算可能位置区域（多边形）直径的算法，并回答以该直径为直径的圆能否覆盖此区域。
2. 已知在某一个检测点处测得的示向度，给出第二个检测点的选择策略，并指出该点可取在什么范围（候选区域）之内。
3. 区域内存在多个干扰源，它们占用互不相同的频道且总个数未知，机器狗从给定起点出发自动定位并逐个清除。要求给出策略与算法，经模拟器演练检验后给出清除比例与平均定位清除时间，并据以完成正式测试。
4. 区域内除全向干扰源外还混有定向干扰源，其个数与发射方向均未知，其余条件与问题三相同。要求给出相应的检测与清除策略及算法。

二、问题分析

四个子问题层层递进。问题一是单次交会定位的几何基础；问题二则将该几何模型进一步用作检测点位置的选择依据，形成单源定位策略；问题三在严格的时间预算下，把单源定位能力嵌入“测向、收敛、清除”的在线决策；问题四基于问题三模型的基础之上，再额外引入定向源方向诊断对决策的影响。

2.1 问题一的分析

本问不宜假设误差分布，而应利用题目给出的误差界，采用最坏情形模型：每个检测点的示向度对应一个由“测量方向 $\pm 1^\circ$ ”张成的角楔。多个检测点的角楔求交，即得干扰源的可能定位区域。由于角楔是凸集，其交仍为凸集（有界时即为凸多边形），因此区域直径可转化为凸多边形顶点间的最远点对问题。求解时，需要先判断交会区域是否有界，再检验以该直径为直径的圆能否覆盖定位区域。整体求解思路可概括为：由误差界构造单点角楔，通过多检测点角楔交会确定凸多边形定位区域，进而将区域直径计算归结为顶点间的最远点对问题；同时辅以有界性判定与圆覆盖检验。

2.2 问题二的分析

问题的关键在于第二个检测点无法由单次观测唯一确定：只掌握 S_1 处的示向度，源点的距离信息完全缺失，故既不能先定位再布点，也不存在一个位置在每次试验中都最好。本文改以“布点后的定位效果之期望”反向衡量布点位置，沿用问题一中“定位区域直径”这一判据，取其对第二次探测点示向度 θ_2 的测量不确定性与源点位置的不确定性的期望为关键判据，既不依赖某一次定位的运气，也无需先知道源点的距离。又因直径的期望只由两次探测的交会几何决定，它只依赖第二个检测点相对第一个检测点的位置，与绝对坐标无关。于是本问归结为以第二检测点对第一检测点的相对位置为决策变量、以 $E[D](a, b)$ 为价值函数的二维寻优问题，其等值线内部有界域即第二个检测点的候选区域。

2.3 问题三的分析

【待填充：给出策略与算法，并补充模拟与实测指标。】

2.4 问题四的分析

【待填充：给出定向源的检测与清除策略、诊断规则及实测结果。】

三、模型假设

为便于建模求解，在合理范围内作如下假设：

1. 示向度误差有界， $|\Delta\theta_i| \leq \varepsilon$ （本题 $\varepsilon = 1^\circ$ ），且同一检测点的误差在观测期间固定不变，只有更换地点才呈现统计规律；故采用有界区间模型（最坏情形），所得区域保证包含真值。
2. 检测点坐标与示向度读数精确已知；示向度以 x 轴正向逆时针为正，取值于 $[0^\circ, 360^\circ)$ 。
3. 干扰源位置在观测与清除期间固定不变，各源频道互不相同且不随时间变化。
4. 测向机与干扰源位于同一平面，区域内无障碍遮挡。
5. 全向源的覆盖角度为 360° ，定向源为定向方向两侧各 90° （含边界），覆盖范围内信号均匀辐射、场强仅随距离衰减。
6. 机器狗沿直线以 5 m/s 匀速行进，忽略转向与加减速耗时；移动中不可检测，单次检测 5 s ，任意两频道切换 1 s 。
7. 与干扰源距离不超过 20 m 时，光学定位（ 3 s ）与激光清除（ 2 s ）必定成功且与覆盖角度无关；距离不超过 5 m 且处于覆盖角度内时无法获得示向度，可直接定位并清除。

四、符号说明

本文使用的主要符号见表 1，未列出的符号在首次出现处说明。

表 1 主要符号说明

符号	含义	单位
$S_i = (x_i, y_i), \theta_i$	第 i 个检测点的坐标与示向度	m, °
$T = (x_T, y_T)$	干扰源（待定位目标）的真实位置	m
$\Delta(\alpha, \beta)$	两方位角之差，归一到 $(-180^\circ, 180^\circ]$	°
ε	示向度误差界（本题 1° ）	°
$\mathcal{R}, V$	定位区域及其顶点集	无
D	定位区域直径	m
M	直径圆圆心（直径中点）	m
A, B	直径端点	m
$S_1, S_2 = (a, b)$	第一、第二个检测点及其相对位置	m
θ_2	第二个检测点处的示向度	°
φ	候选区域极角（自 S_1 的示向度方向向量起）	°
x_{max}	干扰源最大可被有效接收距离	m
$E[D](a, b)$	定位区域直径的期望（第二个检测点的价值函数）	m

五、模型建立与求解

5.1 问题一：示向度交会定位模型

5.1.1 模型建立

设第 i 个检测点为 $S_i = (x_i, y_i)$ ，测得的示向度为 θ_i （ x 轴正向逆时针，取值 $[0^\circ, 360^\circ)$ ），误差界为 $\varepsilon = 1^\circ$ 。干扰源则落在以 S_i 为顶点、关于示向度方向对称张开 2ε 的角楔之内（为避免方位角相减在跨 0° 时出错，记两角之差 $\Delta(\alpha, \beta) = ((\alpha - \beta + 180^\circ) \bmod 360^\circ) - 180^\circ$ ）：

$$\mathcal{W}_i = \{T : |\Delta(\beta(S_i, T), \theta_i)| \leq \varepsilon\}, \quad (1)$$

其中 $\beta(S, T)$ 为自检测点 S 指向干扰源 T 的方位角。 m 次测向给出的定位区域即各角楔之交，如图 1 所示（以 $m = 3$ 为例）。每个角楔可表示为两个闭半平面之交；各角楔对干扰源的约束同时成立，故定位区域为 $\mathcal{R} = \bigcap_{i=1}^m \mathcal{W}_i$ ，它是 $2m$ 个半平面之交，为凸多边形区域，可能有界也可能无界。区域有界时其顶点数不超过 $2m$ ，且每个顶点必是某两条边界直线的交点。记该区域的顶点集为 V 。

若各示向度方向过于接近， $\mathcal{R}$ 沿公共方向无限延伸，定位区域的直径无定义，故须先判定有界性。由于无界是无穷远处的性质，判据只涉及方向：角楔是顶点在 S_i 的锥，若某方向同时落在各角楔张开的方向区间内，则区域内任一点沿该方向平移任意远都不会离开区域，区域因而无界；反之，若区域有界，取其中趋于无穷的一列点单位化，由单位圆的紧性可取收敛子列，而自各 S_i 指向该点列的方位角都趋于该极限方向，角楔又是闭集，故极限方向必落在每个区间内。于是 $\mathcal{R}$ 无界当且仅当各方位角区间 $\Theta_i = [\theta_i - \varepsilon, \theta_i + \varepsilon]$ （跨 0° 按圆周区间处理）存在公共交集，当 $m = 2$ 时即两次测向的方向夹角不超过 2ε 。该判据只需 $O(m)$ 次角度比较，且不依赖检测点坐标。

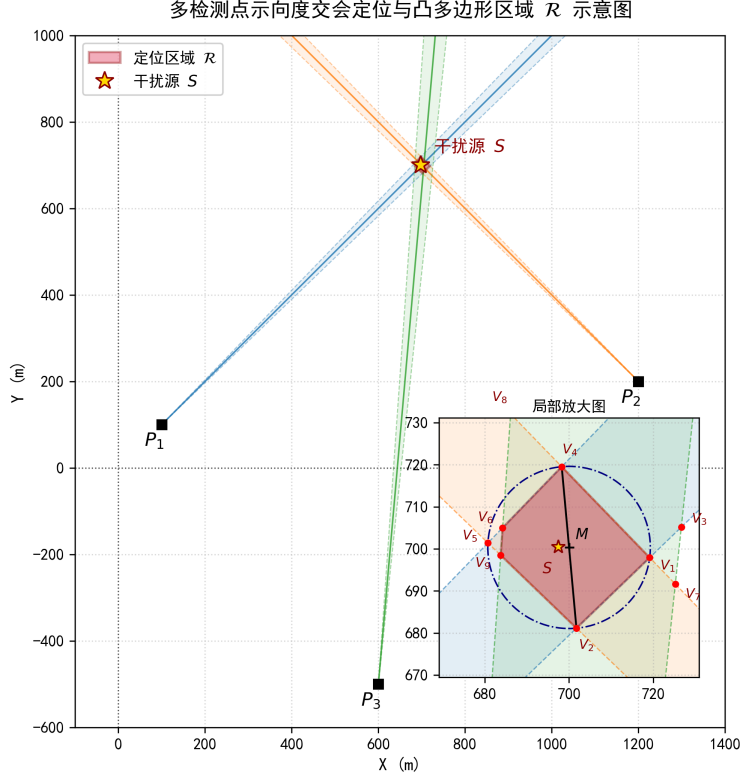


图 1 多检测点示向度交会得到的定位区域；右下角的局部放大图中标出交点 V_i 和直径中点 M

对已判定为有界的区域，凸多边形上任意两点的最大距离必由两个顶点达到。故求直径不必在连续区域上作最优化，只需在顶点集 V 上比较点对距离：

$$D = \max_{P, Q \in \mathcal{R}} \|P - Q\| = \max_{P, Q \in V} \|P - Q\|. \quad (2)$$

设使式 (2) 取到最大值的两个顶点为 A, B ，则直径圆圆心为 $M = (A + B)/2$ 、半径为 $r = D/2$ 。

覆盖与否由圆盘的凸性直接判定： $\mathcal{R} \subseteq$ 该直径圆当且仅当其全部顶点含于圆内，即

$$\max_{P \in V} \|P - M\| \leq D/2. \quad (3)$$

条件不满足时，必有顶点 P 使 $\angle APB < 90^\circ$ 而落在圆外。注意该直径圆的圆心固定为 M ，与圆心可自由选取的最小包围圆不同，故式 (3) 需逐个顶点检验。注意到最远点对可能不唯一，但覆盖判定与取法无关：若某一对最远点 A, B 确定的圆已覆盖 $\mathcal{R}$ ，则 $\mathcal{R}$ 含于半径 $D/2$ 的圆内，而 $\mathcal{R}$ 内任意两点距离不超过 D ，故任何其他最远点对都必对应于该圆上的一条直径，其确定的圆与原圆重合；若某一对不能覆盖，则任何一对也都不能。因此下文任取一对最远点构造直径圆即可。

5.1.2 模型求解算法

算法输入为 m 组“检测点坐标与示向度”及误差界 ϵ ，输出为顶点集 V 、直径 D 、直径端点 A, B 、圆心 M 及覆盖判定，流程如图 2，分四步。

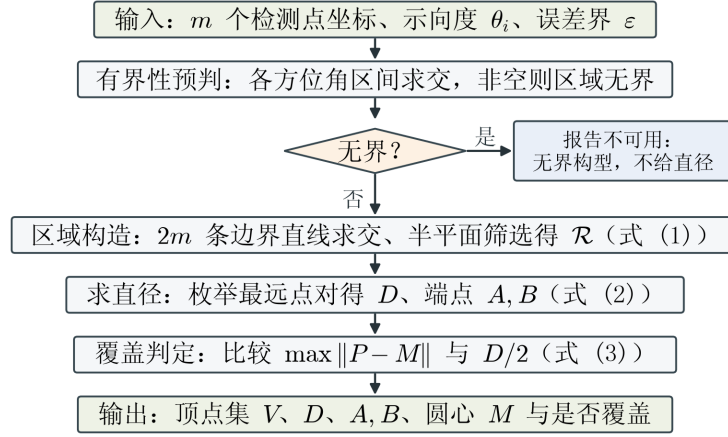


图 2 问题一求解算法流程

第一步（有界性预判）：对 m 个方位角区间 $[\theta_i - \varepsilon, \theta_i + \varepsilon]$ 求交。若交集非空，则定位区域沿该方向无限延伸，直径无定义，算法返回状态 **unbounded**，并提示须更换检测点重测。

第二步（区域构造）：区域的每个顶点都是两条边界直线的交点，故算法枚举每个角楔两条边界射线所在直线（共 $2m$ 条）的两两交点，再用全部 $2m$ 个半平面判据逐一筛选；同一角楔两条边界线的交点即检测点 S_i ，它同样作为候选点保留。随后对候选点集求凸包（包含全部候选点的最小凸多边形，用 Andrew 单调链实现），剔除重复点与近似共线点，得到逆时针排列的顶点表 V 。若候选点集为空，说明各角楔无公共交集（测量数据自相矛盾），算法返回状态 **empty**（无可行定位区域）并终止该例计算；若 $|V| < 3$ ，说明交集退化为点或线段，算法返回状态 **degenerate**（退化）并终止。

第三步（求直径）：按式 (2) 枚举 V 中的顶点对取最大距离，对应两点记为 A, B ，圆心取 $M = (A + B)/2$ ；本题 $m \leq 20$ ，直接枚举即可（至多 $O(m^2)$ 次距离计算）。为交叉验证，本文还采用旋转卡壳算法（凸多边形直径的线性时间算法）复算 D ，两法结果一致。

第四步（覆盖判定）：按式 (3) 计算 $\max_{P \in V} \|P - M\|$ ，判定条件为

$$\max_{P \in V} \|P - M\| \leq D/2 + \tau, \quad (4)$$

其中绝对容差 $\tau = 10^{-9}$ m 用于容忍相切构型的浮点误差，避免相切构型被误判为不覆盖；判定不覆盖时，输出取到该最大值的顶点作为越界位置。

为保证结果不依赖具体实现，程序另有一套独立等价的区域构造算法：自一个大正方形出发，用 Sutherland–Hodgman 算法逐个半平面裁剪多边形。两者给出的顶点集逐点一致（最大偏差不超过 10^{-9} m），状态名与原因说明见附录 B 的 `analyse()`。

5.1.3 求解结果与分析

使用程序构造五组良态算例（检测点分布在干扰源四周，方位张角 $180^\circ \sim 249^\circ$ 、最小方位间隔不小于 25° 、与干扰源距离 $160 \sim 1300$ m），结果见表 2；五组算例的定位区域直径均在 $10.59 \sim 25.64$ m 之间，即两次以上交会已能把候选范围从测向距离的百米量级收缩到数十米以内。表中同时给出覆盖判定，其两种结果的对照见图 3：左图（算

例 A_1) 的定位区域两个顶点都落在直径圆上, 最远顶点到圆心 M 的距离恰为 $D/2$ (比值 1.0000), 此时直径圆就是该区域的最小包围圆; 右图 (算例 A_2) 中, 橙色虚线标出的顶点 P 落在圆外, $|PM| = 10.29 \text{ m}$ 比 $D/2 = 9.31 \text{ m}$ 大 0.98 m (比值 1.1055), 直径圆覆盖判定为“否”。

表 2 问题一算例的计算结果

算例	m	顶点数	$D \text{ (m)}$	$D/2 \text{ (m)}$	$\max \ P - M\ \text{ (m)}$	比值	覆盖
A_1	6	6	17.64	8.82	8.82	1.0000	是
A_2	5	5	18.61	9.31	10.29	1.1055	否
A_3	4	5	25.64	12.82	13.75	1.0722	否
A_4	5	4	21.12	10.56	10.56	1.0000	是
A_5	6	5	10.59	5.30	5.30	1.0000	是

可见五组算例中有两组 (A_2 、 A_3) 的直径圆无法覆盖定位区域, 越界顶点到 M 的距离分别比 $D/2$ 大 0.98 m 与 0.93 m , 超出 10.55% 与 7.22%; 这两组算例的区域均为斜五边形, 由于 A_2 中两个顶点距离较近, 故可以近似为四边形, 而越界顶点所对直径形成的张角小于 90° (图 3 右图中圈出的顶点张角为 84.2°), 因而落在圆外。

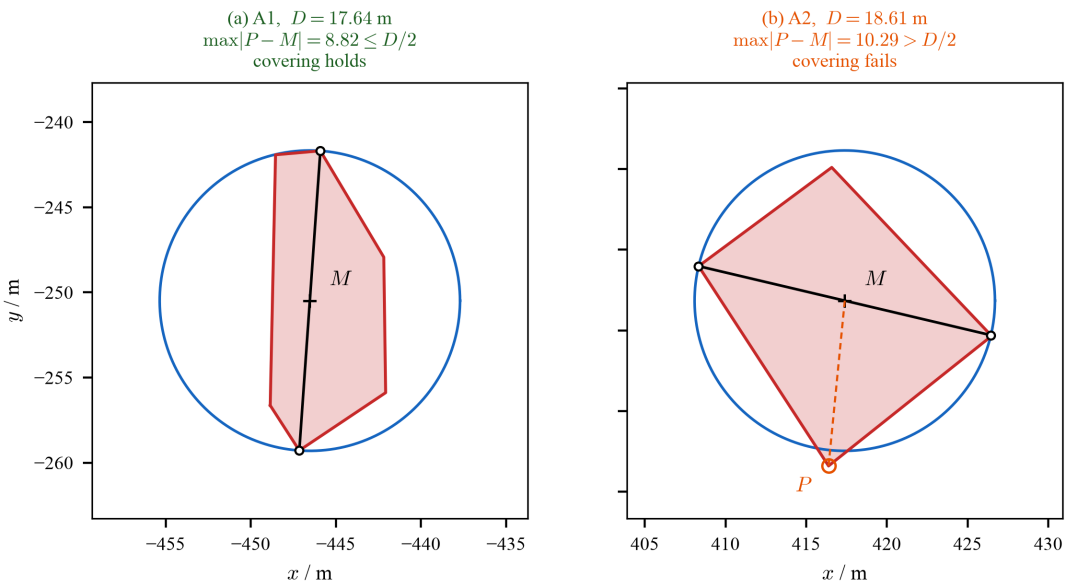


图 3 直径圆覆盖判定的两种结果: (a) 覆盖成立; (b) 覆盖不成立

表 3 四组随机扫描的覆盖失败统计 (固定种子, 理想/实际构型逐例配对)

构型	检测点数	试验数	有界数	失败数	失败率	最坏比值	需放大约
理想	2	20000	20000	231	1.16%	1.0336	3.4%
理想	3 ~ 6	4000	3472	113	3.25%	1.0304	3.0%
实际	2	20000	19996	246	1.23%	1.0346	3.5%
实际	3 ~ 6	4000	3472	993	28.60%	1.6876	68.8%

注: 理想构型指各示向度精确指向源点; 实际构型指在上一批构型上叠加与算例生成同分布的 $\pm \varepsilon$ 测向误差; “需放大约”为最坏比值减 1。

为说明这种失败并非偶然，本文以固定随机种子作四组扫描（表 3）。叠加测向误差后，3 ~ 6 个检测点的覆盖失败率由 3.25% 升至 28.60%、最坏时直径圆需放大约 69%，且失败率随检测点数递增；可见只要各次测向的误差方向不一致，定位区域便会偏离以源点为公共交点的对称形状，直径圆随之失效，表 2 中 A_2 、 A_3 的失败即属于这一情形。本节数值的数据清单与复现命令见附录 A。

5.2 问题二：第二检测点的选择模型

5.2.1 模型建立

记号与概率描述

S_1 置于原点，其示向度方向取为 x 轴正方向； S_2 置于 (a, b) 。已知 S_1 的示向度后，源点 T 可能位置 (x_T, y_T) 服从一个条件概率分布

$$T \sim P_T(T), \quad (5)$$

而给定源点位置与 S_2 的位置， θ_2 的读数由于误差亦不能确定，近似服从

$$\theta_2 \sim P_\theta(\theta_2 | T, S_2). \quad (6)$$

位形参数 (θ_2, a, b) 一经确定，定位区域直径便由几何关系唯一确定给出：

$$D = D(\theta_2, a, b). \quad (7)$$

于是 D 服从由几何参量 (a, b) 控制的概率分布，第二个检测点的选择问题即化为求如下二维价值函数：

$$E[D](a, b) = \iint d^2 \vec{r}_T P_T(T | S_1) \int d\theta_2 D(\theta_2, a, b) P_\theta(\theta_2 | T, S_2), \quad (8)$$

$E[D](a, b)$ 越小，在该点放置第二个检测点所得定位平均效果越好。这即为问题二求解的关键衡量指标。

5.2.2 理论求解

(a) 理论推导

式 (8) 含内外两次共计三重概率积分，直接使用不便，故作以下四条近似：

1. S_2 距 S_1 的距离 R 不太小。该近似的合理性并非天然成立： S_2 距 S_1 过近时两视线的交角趋于零，所得定位区域必然很大，这是我们不希望的，故 R 较小的区域本来就不在选择范围之内，而本问关心的待选区域附近该近似较好。
2. 忽略源点纵坐标，认为其恰好落在 x 轴上，其位置只由一个参数 x_T 描述。当 R 不小时源点纵坐标 y_T 是小量，将分布对 y_T 作展开：

$$P_\theta(\theta_2 | x_T, y_T, a, b) \sim P_\theta(\theta_2 | x_T, 0, a, b) + \frac{d}{dy_T} P_\theta \Big|_{y_T=0} y_T + o(y_T) \quad (9)$$

而在第一个概率积分中区间关于 $y_T = 0$ 对称，一阶项相互抵消，故 y_T 对 $E[D]$ 的影响是高阶小量。

3. 考虑 y_T 小量近似后所得零阶项 $P_\theta(\theta_2 | x_T, 0, a, b)$ 在扇形区间内积分，可认为先验权重满足 $P_T(x_T | S_1) \propto x_T$ ，归一化后得

$$P_T(x_T | S_1) = \frac{2x_T}{x_{max}^2}, \quad x_T \in [0, x_{max}], \quad (10)$$

其中 x_{max} 为干扰源的最大可被有效接收距离，在本问中可直接取为 1500m。

4. 与第 2 条同理，由于探测点二实际测得方向角在理论值 $\theta_{2,ideal}$ 左右 1° 区间内均匀分布，偏离值为小量认为其分布近似 $\theta_2 \sim \delta(\theta_2 - \theta_{2,ideal})$ ，即等效直接认为第二次探测所得方位正对源点。

四条近似把价值函数精简为单个一维积分：

$$E[D](a, b) = \int_0^{x_{max}} dx_T P_T(x_T | S_1) D(x_T, a, b). \quad (11)$$

(b) 定位区域直径的解析推导

近似取定 θ_2 正对源点后，两条探测视线在源点处的交角 α 满足

$$\sin \alpha = \frac{|b|}{r_{2T}}, \quad r_{2T} = \sqrt{(x_T - a)^2 + b^2}, \quad (12)$$

图 4 为一个特定情况下的两个探测点探测示意图，并取源点 T 、第二检测点 S_2 ；图中蓝、黄两个区域分别为两次探测各自的定位区域， $\angle S_1 T S_2 = \alpha$ 为两视线在源点处的交角，右上角为两区域相交所得的定位区域 $ABCD$ 的局部放大图。所得定位区域为局部图中的红色近似平行四边形 $ABCD$ ：该区域的直径即对角线 $|BD|$ 。

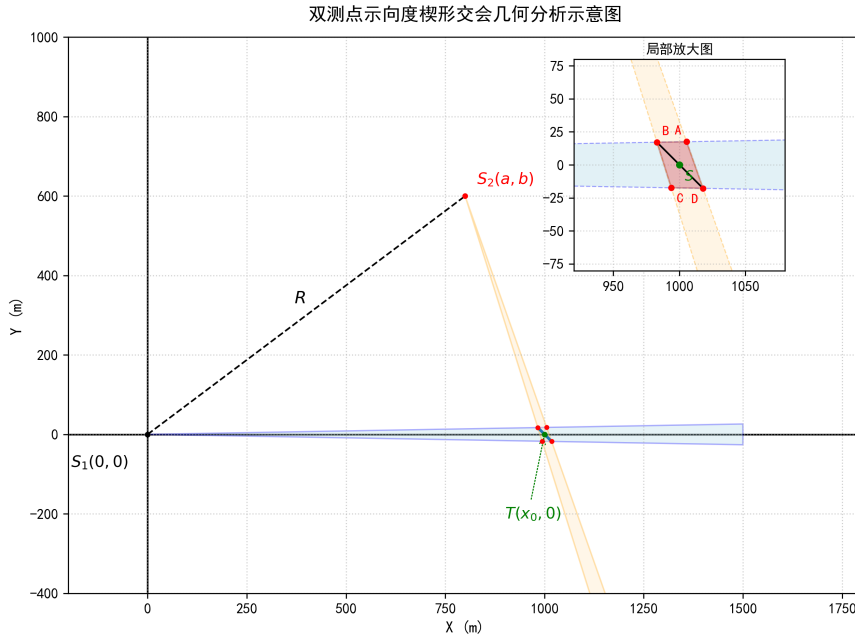


图 4 两探测束交会产生定位区域示意图

本文探测束张角极小 ($\Delta\theta = 2\varepsilon = 2^\circ \approx 0.0349 \text{ rad} \ll 1$)，故在源点处两次探测束的横向宽度分别为

$$w_1 = x_T \Delta\theta, \quad w_2 = r_{2T} \Delta\theta = \sqrt{(x_T - a)^2 + b^2} \Delta\theta. \quad (13)$$

由式 (12) 及 $(x_T - a)/r_{2T}$ 的符号, 交角还满足

$$\cos \alpha = \frac{x_T - a}{r_{2T}} = \frac{x_T - a}{\sqrt{(x_T - a)^2 + b^2}}. \quad (14)$$

其对角线长度由两组对边的宽度与夹角 α 完全确定:

$$D(x_T, a, b) = \frac{\sqrt{w_1^2 + w_2^2 + 2w_1w_2 \cos \alpha}}{\sin \alpha}. \quad (15)$$

该式有直接的几何解释: 下标 1、2 的两项分别对应两次探测各自的束宽, 交叉项 $2w_1w_2 \cos \alpha$ 则来自两组对边之间的夹角。

把式 (13)、(12) 与 (14) 代入式 (15), 即可消去 w_1, w_2, α , 将直径完全显式化为 a, b, x_T 的函数。分子先提出公因子 $r_{2T}\Delta\theta$, 被开方式化为 $x_T^2 + r_{2T}^2 + 2x_T(x_T - a) = 4x_T^2 - 4ax_T + a^2 + b^2$, 于是

$$\begin{aligned} D(x_T, a, b) &= \frac{\Delta\theta r_{2T}}{|b|} \sqrt{x_T^2 + r_{2T}^2 + 2x_T(x_T - a)} \\ &= \frac{\Delta\theta}{|b|} \sqrt{(x_T - a)^2 + b^2} \sqrt{(2x_T - a)^2 + b^2}, \end{aligned} \quad (16)$$

其中第一个根式即式 (12) 中的 $r_{2T} = |S_{2T}|$, 第二个根式是源点到点 $(a/2, -b/2)$ 之距离的两倍。定位区域直径由这两个几何量的乘积与 $|b|$ 之比给出。

式 (16) 的合理性可由两个极限情形检验: 当第二个检测点正落在第一个视线方向附近 ($a \rightarrow x_T, b \rightarrow 0$) 时交角趋于 90° , 此时 $D \rightarrow \Delta\theta x_T = w_1$, 恰为定位区域沿垂向的宽度; 当 $b \rightarrow 0$ 且 $x_T \neq a$ 时两视线共线、 $\sin \alpha \rightarrow 0, D \rightarrow \infty$, 与式 (12) 的退化判据一致。

代入 $\Delta\theta = 2^\circ = \pi/90 \text{ rad}$, 得直径函数的解析表达式

$$D(x_T, a, b) = \frac{\pi}{90|b|} \sqrt{(x_T - a)^2 + b^2} \sqrt{(2x_T - a)^2 + b^2}. \quad (17)$$

再代回式 (11) 并利用式 (10), 即得可直接用于数值积分与优化的目标函数

$$E[D](a, b) = \frac{\pi}{45 x_{max}^2 |b|} \int_0^{x_{max}} x_T \sqrt{[(x_T - a)^2 + b^2][(2x_T - a)^2 + b^2]} dx_T. \quad (18)$$

引入无量纲变量 $u = x_T/x_{max} \in [0, 1]$ 与 $\hat{a} = a/x_{max}, \hat{b} = b/x_{max}$, 式 (18) 还可写成

$$E[D](a, b) = \frac{\pi x_{max}}{45 |\hat{b}|} \int_0^1 u \sqrt{[(u - \hat{a})^2 + \hat{b}^2][(2u - \hat{a})^2 + \hat{b}^2]} du, \quad (19)$$

即 $E[D]$ 只依赖 x_{max} 与无量纲位形 $(\hat{a}, \hat{b})$, 与绝对尺度无关。

(c) 价值函数的闭合解

价值函数的不定积分核为

$$\int x_T \sqrt{[(x_T - a)^2 + b^2][(2x_T - a)^2 + b^2]} dx_T. \quad (20)$$

作平移 $t = x_T - a/2$ (即 $x_T = t + a/2$), 两个二次因子化为关于 t 的偶函数

$$(x_T - a)^2 + b^2 = \left(t - \frac{a}{2}\right)^2 + b^2, \quad (2x_T - a)^2 + b^2 = 4t^2 + b^2, \quad (21)$$

即根号内是 t 的**四次多项式**（偶四次）。因此式 (20) 的原函数属于椭圆积分类型，不能由初等函数显示表示。

5.2.3 数值方法求解

111111111111111111111111111111111 这一段需要重写来和代码模拟保持一致
 第一步建立标准坐标系： S_1 置于原点，其示向度方向取为 x 轴正向， $S_2 = (a,b)$ ；于是几何量 $|b|$ 、 r_{2T} 、 α 都可用 a, b, x_T 写出。

第三步求期望：按重心法在 $[0, x_{max}]$ 上取 n 个分点 $x_T^{(k)}$ ，以 $w_k \propto x_T^{(k)}$ （即式 (10)）加权：

数值上该式对分点数收敛很快, n 取 200 ~ 800 已稳定到 0.01 m 以内。

5.2.4 求解结果与分析

图中的空白区域是不可行区，或为 $|S_2S_1| > x_{max}$ （两次探测不可能同时有效接收），或为两视线近似共线使 D 无界。图 5(b) 把同一信息画成候选区域：红色点划线为 S_1 的示向度方向 ψ_1 ，正方形为按 $x_T = 1100$ m 画出的示意源点，深、浅两块分别为 $E[D] \leq 1.05E[D]_{\min}$ 与 $E[D] \leq 1.20E[D]_{\min}$ 的解集。

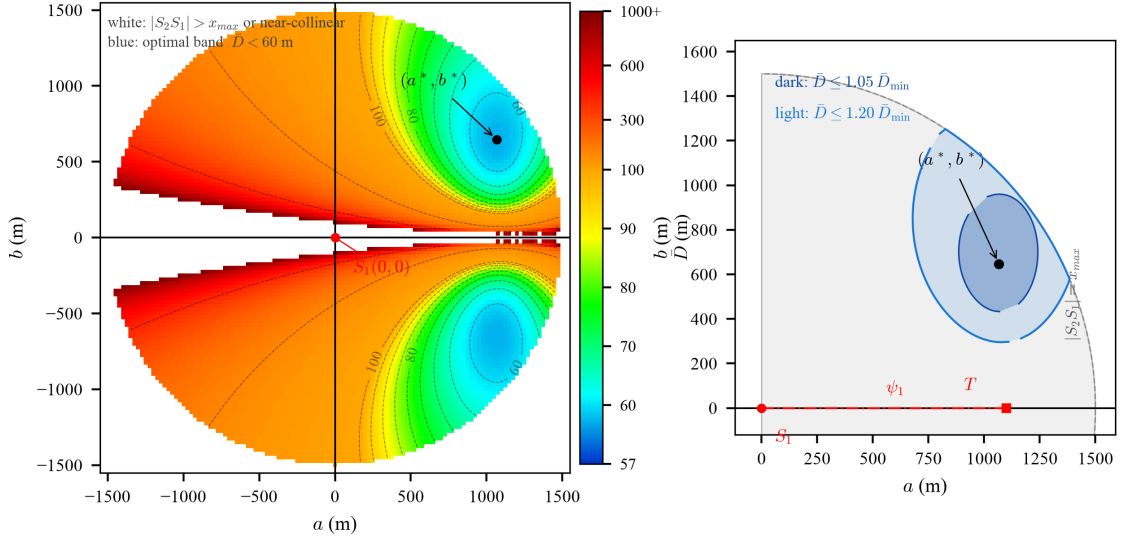


图 5 期望定位区域直径的等值图与第二个检测点的候选区域

线被拉成极窄的长条， $E[D]$ 达 10^3 m 量级；只有把第二点移到视线的一侧、使两次探测的视线夹角足够大（约 $55^\circ \sim 80^\circ$ ）时， $E[D]$ 才降到 57 m 量级，即交会定位的效果由两次探测的**视线交角**主导，而不是由第二点离源点的远近主导。

表 4 不同第二检测点位置的期望定位区域直径

布点方案	a (m)	b (m)	$E[D]$ (m)
S_2 与 S_1 重合	0	0	无界（视线平行）
沿视线前移 500 m	499.9	8.7	2647
沿视线前移 1000 m	999.8	17.5	5604
最优位置 (a^*, b^*)	1066.0	646.4	57.2
(1000, 600)	1000	600	57.8
(1000, 700)	1000	700	57.7
(1100, 600)	1100	600	57.4
(1200, 700)	1200	700	58.9
正对 S_1 侧移 700 m	0	700	141.7
(500, 700)	500	700	83.0

求解得到的最优点是 $(\hat{a}^*, \hat{b}^*) \approx (0.71, 0.43)$ ，即

$$a^* \approx 0.71 x_{max}, \quad b^* \approx 0.43 x_{max}, \quad (23)$$

在本例参数下为 $(a^*, b^*) \approx (1066, 646)$ m， $|S_1 S_2| \approx 1247$ m，即最优位置在 S_1 视线正前方约 1066 m、再向侧向偏约 646 m，视线夹角约 31° 处，此时

$$E[D]_{\min} = E[D](a^*, b^*) \approx 57.2 \text{ m}, \quad (24)$$

与目标区域半径（1800 m）相比不到 1/30。

答案所要求的是范围而不是一个点，故取相对容差 δ ，将 $E[D] \leq (1 + \delta)E[D]_{\min}$ 的解集作为第二检测点的候选区域。表 5 给出两个容差下的数值结果，其中 φ 自 S_1 的视

线方向量起。可以看出：容差收紧到 5% 时，可选范围已收缩成一条很窄的环扇区；放宽到 20% 时，可行范围几乎铺满视线前方 $R \geq 0.62x_{max}$ 的整个扇面，说明 $E[D]$ 在最优带附近相当平坦，允许现场按可达性在相当大的范围内取舍。

表 5 不同容差下的候选区域

容差 δ	$R = S_2 S_1 $ 范围	φ 范围	说明
5%	[1071, 1484] m	[22.0°, 43.1°]	最优带
20%	[923, 1500] m	[15.0°, 57.0°]	候选区域

取便于现场执行的矩形外包络，第二个检测点可取在以 S_1 为极点、其示向度方向 ψ_1 为极轴的环扇区 $\varphi \in [15^\circ, 57^\circ]$ 、 $R \in [0.62x_{max}, x_{max}]$ 之内，即视线前方约 930 ~ 1500 m、且偏离视线约 $15^\circ \sim 57^\circ$ 的区域；该外包络中约 69% 的点满足 $E[D] \leq 1.2E[D]_{\min}$ ，最优带 ($\leq 1.05E[D]_{\min}$) 则集中在 $\varphi \approx 22^\circ \sim 43^\circ$ 、 $R \approx 0.71 \sim 0.99x_{max}$ 。实际布点时优先取 $(a^*, b^*) \approx (0.71x_{max}, 0.43x_{max})$ ，并尽量使两次探测的视线夹角落在 $55^\circ \sim 80^\circ$ 之间。

该结果与模型的四条近似自洽：其一， $E[D]$ 与 $\Delta\theta$ 成正比，最优点处 $E[D]/\Delta\theta \approx 3279$ m 在 $\Delta\theta$ 由 0.5° 变到 4° 时几乎不变，说明候选区域不依赖误差界的具体取法；其二， $b \rightarrow 0$ （两点与源点近似共线）时 $E[D] \rightarrow \infty$ ，这与“ S_2 距 S_1 太近、所得定位区域必然较大”的建模前提一致，也说明所作近似并不掩盖几何的退化。两条求解路线（按式 (22) 对式 (17) 作加权求积、以及直接由四边形顶点两两距离取最大者）给出的最小值相差不到 0.2%，最优位置相差不到 20 m，二者互为独立验证。

5.3 问题三：多源定位与清除策略

5.3.1 模型建立

【待填充：给出该问的模型与求解方法。】

5.3.2 求解与结果

【待填充：给出该问的求解结果与实测指标。】

5.4 问题四：含定向源时的定位与清除模型

5.4.1 模型建立

【待填充：给出该问的模型与求解方法。】

5.4.2 求解与结果

【待填充：给出该问的求解结果与分析。】

六、模型评价与改进

6.1 灵敏度分析

【待填充：对模型中的关键参数作灵敏度分析。】

6.2 模型优点

【待填充：完成问题二至四后统一总结四个模型的优点。】

6.3 模型缺点

【待填充：同上，统一总结缺点。】

6.4 改进方向

【待填充：统一给出改进方向。】

参考文献

- [1] 作者. 无线电测向与定位技术 [M]. 出版社, 年份.
- [2] 作者. 论文题目 [J]. 期刊名, 年份, 卷 (期): 页码.
- [3] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛论文格式规范 (2026 年修订稿) .

附录 A 支撑材料文件列表

本论文支撑材料所包含的文件如下（路径均位于 B_locator/ 目录下）。问题一、问题二的算例与全部数值结果均由清单中的脚本运行生成，问题一插图已定版，随材料一并提供；数值计算仅依赖 NumPy，全过程离线。

1. Q1/PAPER-DATA.md: 问题一全部数值的数据清单、来源字段与一键复现命令（正文表 2、表 3 与摘要数值均可据此复核）。
2. Q1/p1_intersection.py: 问题一求解库，含定位区域构造、直径与覆盖判定（附录 B）。
3. code/main.py: 问题三、四主程序，负责模拟器交互与搜索清除。
4. Q1/p1_experiments.py: 算例生成、逐例求解与四组随机扫描（正文表 2、表 3 的数值来源）。
5. Q1/p1_selftest.py: 自检脚本，含真值包含、凸性、直径双算法、两套区域实现交叉校验与逐字节复现校验。
6. Q1/cases/p1_case01~05.csv（含 .truth.json）: 五组算例与真值（固定种子生成，真值仅自检用）。
7. Q1/p1_results.jsonl、Q1/p1_summary.json: 逐例结果与汇总统计（含四组随机扫描）。
8. figures/ 下的 3 张问题一插图（定版；Q1/ 下有同名副本，逐字节一致）：图 1、图 2、图 3。
9. Q2/src/q2_solution.py: 问题二源程序，期望直径、寻优与候选区域（附录 C）。
10. Q2/figures/make_q2_figures.py: 问题二插图生成脚本。
11. Q2/src/results/q2_summary.json: 问题二最优点、等值区域与自检汇总（本文数值来源）。
12. figures/p2-expected-diameter.png: 期望直径等值图与候选区域（图 5）。

13. figures/p2-wedge-geometry.png: 两探测束交会与定位区域直径构造 (图 4)。
14. Q2/Q2modeling.md 等: 问题二建模手稿、参考热力图与 Mathematica 校验脚本。

附录 B 问题一主要源程序

问题一的完整求解源程序全文见下; 定位区域按“顶点枚举 + 半平面筛选 + 凸包规范化”构造, 另有 Sutherland–Hodgman 裁剪的等价实现用于交叉校验。算例生成、四组随机扫描与不变量自检所用的脚本及其用途见附录 A。

Listing 1: 问题一: 定位区域、区域直径与直径圆覆盖判定 (Python 3)

```
# -*- coding: utf-8 -*-
"""
    m x y theta x
    [0,360) |e| <= BEARING_ERR = 1

    R = W_i W_i S_i 2*BEARING_ERR
        R 2m
    max|P-M| D/2 M

    p1_experiments.py
    p1_selftest.py
"""
from __future__ import annotations
import csv
import json
import math
import os

import numpy as np

BEARING_ERR = 1.0 #
ANG_TOL = 1e-9 #
DIST_TOL = 1e-6 #
CLIP = 1e7 #
DIAM_BRUTE_LIMIT = 128 #

# -----
#
# -----

def bearing(p, q):
    """p q [0,360) """
    return math.degrees(math.atan2(q[1] - p[1], q[0] - p[0])) % 360.0

def ang_diff(a, b):
    """a-b (-180,180]"""
    return (a - b + 180.0) % 360.0 - 180.0

def wedge_halfplanes(p, theta, err=BEARING_ERR):
    """ W = {X : |ang(X-p) - theta| <= err} a · X + b <= 0"""
    p = np.asarray(p, float)
    tm = math.radians((theta - err) % 360.0)
    tp = math.radians((theta + err) % 360.0)
    dm = np.array([math.cos(tm), math.sin(tm)])
    dp = np.array([math.cos(tp), math.sin(tp)])
```

```

# cross(dm, X-p) >= 0 <=> a1 · X + b1 <= 0
a1 = np.array([dm[1], -dm[0]])
b1 = float(-dm[1] * p[0] + dm[0] * p[1])
# cross(dp, X-p) <= 0 <=> a2 · X + b2 <= 0
a2 = np.array([-dp[1], dp[0]])
b2 = float(dp[1] * p[0] - dp[0] * p[1])
return [(a1, b1), (a2, b2)]

def region_is_unbounded(svds, err=BEARING_ERR):
    """
    """
    for th in svds:
        for cand in ((th - err) % 360.0, (th + err) % 360.0):
            if all(abs(ang_diff(cand, t)) <= err + ANG_TOL for t in svds):
                return cand
    return None

def _line_intersection(P, u, Q, v):
    """
    P + t · u    Q + s · v
    """
    det = u[0] * (-v[1]) - (-v[0]) * u[1]
    if abs(det) < 1e-14:
        return None
    r = Q - P
    t = (r[0] * (-v[1]) - (-v[0]) * r[1]) / det
    return P + t * u

def convex_hull(P):
    """Andrew monotone chain"""
    def half(pts):
        st = []
        for p in pts:
            while len(st) >= 2 and _cross2(st[-1] - st[-2], p - st[-2]) <= 0:
                st.pop()
            st.append(p)
        return st
    P = P[np.lexsort((P[:, 1], P[:, 0]))]
    lo = half(P)
    hi = half(P[::-1])
    return np.array(lo[:-1] + hi[:-1])

def _cross2(u, v):
    return float(u[0] * v[1] - u[1] * v[0])

def _dedup_hull(cand):
    """
    + (P, "empty"|"degenerate"|"polygon")"""
    if len(cand) == 0:
        return np.zeros((0, 2)), "empty"
    P = np.array(cand, float)
    P = P[np.lexsort((P[:, 1], P[:, 0]))]
    ded = [P[0]]
    for v in P[1:]:
        if np.linalg.norm(v - ded[-1]) > 1e-7:
            ded.append(v)
    P = np.array(ded)
    if len(P) < 3:
        return P, "degenerate"
    H = convex_hull(P)
    if len(H) < 3:

```

```

        return H, "degenerate"
    return H, "polygon"

# -----
#           1           +
# -----
def region_by_vertices(pts, svds, err=BEARING_ERR, tol=DIST_TOL):
    """
        2m

    R                                     +

    """
    if len(pts) == 0:
        return np.zeros((0, 2)), "empty"
    H, lines = [], []
    for p, th in zip(pts, svds):
        H += wedge_halfplanes(p, th, err)
        for s in (-1.0, 1.0):
            a = math.radians((th + s * err) % 360.0)
            lines.append((np.asarray(p, float), np.array([math.cos(a), math.sin(a)])))
    cand = []
    n = len(lines)
    for i in range(n):
        for j in range(i + 1, n):
            X = _line_intersection(lines[i][0], lines[i][1], lines[j][0], lines[j][1])
            if X is None or not np.all(np.isfinite(X)):
                continue
            if all(float(a @ X + b) <= tol for (a, b) in H):
                cand.append(X)
    return _dedup_hull(cand)

# -----
#           2
# -----
def clip_halfplane(poly, a, b, tol=DIST_TOL):
    """Sutherland-Hodgman {X : a · X + b <= 0} """
    if not poly:
        return []
    out = []
    n = len(poly)
    for i in range(n):
        P = poly[i]
        Q = poly[(i + 1) % n]
        fp = float(a @ P + b)
        fq = float(a @ Q + b)
        if fp <= tol:
            out.append(P)
        if (fp < -tol and fq > tol) or (fp > tol and fq < -tol):
            t = fp / (fp - fq)
            out.append(P + t * (Q - P))
    ded = []
    for v in out:
        if not ded or np.linalg.norm(v - ded[-1]) > 1e-9:
            ded.append(v)
    if len(ded) > 1 and np.linalg.norm(ded[0] - ded[-1]) < 1e-9:
        ded.pop()
    return ded

def region_by_clipping(pts, svds, err=BEARING_ERR, box=CLIP, tol=DIST_TOL):

```

```

"""
poly = [np.array([-box, -box], float), np.array([box, -box], float),
        np.array([box, box], float), np.array([-box, box], float)]
for p, th in zip(pts, svds):
    for (a, b) in wedge_halfplanes(p, th, err):
        poly = clip_halfplane(poly, a, b, tol)
    if not poly:
        return np.zeros((0, 2)), "empty"
P = np.array(poly, float)
if len(P) < 3:
    return P, "degenerate"
if float(np.max(np.abs(P))) > box * 0.5:
    return P, "unbounded"
H = convex_hull(P)
return (H if len(H) >= 3 else P), "polygon"

def region_from_bearings(pts, svds, err=BEARING_ERR):
    """
    (, status) status empty / unbounded / degenerate / bounded """
    P, st = region_by_vertices(pts, svds, err)
    if st == "empty":
        return P, "empty"
    rec = region_is_unbounded(list(svds), err) if len(svds) else None
    if rec is not None:
        return P, "unbounded"
    if st == "degenerate":
        return P, "degenerate"
    return P, "bounded"

# -----
#
# -----

def _diameter_brute(P):
    n = len(P)
    best, pair = -1.0, (P[0], P[1])
    for i in range(n - 1):
        d = np.linalg.norm(P[i + 1:] - P[i], axis=1)
        k = int(np.argmax(d))
        if float(d[k]) > best:
            best, pair = float(d[k]), (P[i], P[i + 1 + k])
    return best, pair

def rotating_calipers(P):
    """
    O(n) P """
    n = len(P)
    if n < 3:
        return diameter(P, brute_limit=2)
    k = 1
    best, pair = -1.0, (P[0], P[1])
    for i in range(n):
        i2 = (i + 1) % n
        e = P[i2] - P[i]
        while True:
            k2 = (k + 1) % n
            if abs(_cross2(e, P[k2] - P[i])) > abs(_cross2(e, P[k] - P[i])):
                k = k2
            else:
                break
        for j in (k, (k + 1) % n):
            d = float(np.linalg.norm(P[i] - P[j]))

```

```

        if d > best:
            best, pair = d, (P[i], P[j])
        d = float(np.linalg.norm(P[i2] - P[j]))
        if d > best:
            best, pair = d, (P[i2], P[j])
    return best, pair

def diameter(P, brute_limit=DIAM_BRUTE_LIMIT):
    """
        (D, (A, B))"""
    n = len(P)
    if n == 0:
        return 0.0, None
    if n == 1:
        return 0.0, (P[0], P[0])
    if n == 2:
        return float(np.linalg.norm(P[0] - P[1])), (P[0], P[1])
    if n <= brute_limit:
        return _diameter_brute(P)
    return rotating_calipers(P)

def min_enclosing_circle(P):
    """
        (center, radius) 1/2/3 """
    P = np.asarray(P, float)
    n = len(P)
    if n == 0:
        return None, 0.0
    if n == 1:
        return P[0].copy(), 0.0
    cands = [(P[i].copy(), 0.0) for i in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            c = 0.5 * (P[i] + P[j])
            cands.append((c, float(np.linalg.norm(P[i] - c))))
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                A, B, C = P[i], P[j], P[k]
                d = 2.0 * (A[0] * (B[1] - C[1]) + B[0] * (C[1] - A[1])
                           + C[0] * (A[1] - B[1]))
                if abs(d) < 1e-12:
                    continue
                aa, bb, cc = float(A @ A), float(B @ B), float(C @ C)
                ux = (aa * (B[1] - C[1]) + bb * (C[1] - A[1]) + cc * (A[1] - B[1])) / d
                uy = (aa * (C[0] - B[0]) + bb * (A[0] - C[0]) + cc * (B[0] - A[0])) / d
                c = np.array([ux, uy])
                cands.append((c, float(np.linalg.norm(A - c))))
    best = None
    for c, r in cands:
        if all(float(np.linalg.norm(P[i] - c)) <= r + 1e-9 for i in range(n)):
            if best is None or r < best[1]:
                best = (c, r)
    return best

def region_area(P):
    n = len(P)
    if n < 3:
        return 0.0
    return abs(0.5 * float(np.sum(P[:, 0] * np.roll(P[:, 1], -1)
                                     - np.roll(P[:, 0], -1) * P[:, 1])))

```

```

# -----
#
# -----
def analyse(pts, svds, err=BEARING_ERR):
    """
    """
    P, status = region_from_bearings(pts, svds, err)
    base = {"n_bearings": len(pts), "status": status}
    if status == "empty":
        base["reason"] = (
            "
            ")
        return base
    if status == "unbounded":
        rec = region_is_unbounded(list(svds), err)
        base["reason"] = (
            "
            = +inf
            ")
        base["recession_direction_deg"] = None if rec is None else round(float(rec), 4)
        base["n_vertices"] = int(len(P))
        base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P]
        return base
    if status == "degenerate" or len(P) < 3:
        base["status"] = "degenerate"
        base["reason"] = "
        "
        base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P]
        return base

    D, (A, B) = diameter(P)
    c = 0.5 * (A + B)
    maxr = float(max(np.linalg.norm(v - c) for v in P))
    mc, mr = min_enclosing_circle(P)
    base.update({
        "n_vertices": int(len(P)),
        "vertices": [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P],
        "diameter_m": round(D, 4),
        "diameter_endpoints": [[round(float(A[0]), 4), round(float(A[1]), 4)],
                               [round(float(B[0]), 4), round(float(B[1]), 4)]],
        "circle_center": [round(float(c[0]), 4), round(float(c[1]), 4)],
        "circle_radius_m": round(D / 2.0, 4),
        "max_vertex_dist_m": round(maxr, 4),
        "diameter_circle_covers": bool(maxr <= D / 2.0 + 1e-9),
        "min_enclosing_center": [round(float(mc[0]), 4), round(float(mc[1]), 4)],
        "min_enclosing_radius_m": round(float(mr), 4),
        "min_circle_over_half_diameter": round(float(mr) / (D / 2.0), 6),
        "area_m2": round(region_area(P), 4),
    })
    return base

def run_case_csv(path, truth_path=None, err=BEARING_ERR):
    """
    CSV x_m,y_m,svd_deg JSON analyse
    """
    with open(path, encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    pts = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
    svds = [float(r["svd_deg"]) for r in rows]
    res = analyse(pts, svds, err)
    res["n_bearings"] = len(rows)
    if truth_path and os.path.exists(truth_path):
        with open(truth_path, encoding="utf-8") as f:
            truth = json.load(f)
        G = truth["source_xy"]
        res["truth_xy"] = G

```

```

if "circle_center" in res:
    res["center_error_vs_truth_m"] = round(math.dist(res["circle_center"], G), 3)
if "min_enclosing_center" in res:
    res["mincircle_error_vs_truth_m"] = round(
        math.dist(res["min_enclosing_center"], G), 3)
#
H = []
for p, th in zip(pts, svds):
    H += wedge_halfplanes(p, th, err)
res["truth_inside_region"] = bool(all(a @ np.asarray(G, float) + b <= 1e-6
                                     for (a, b) in H))

return res

```

附录 C 问题二主要源程序

问题二的完整源程序（支撑材料 Q2/src/q2_solution.py）见下，仅保留与正文式 (11)–(18) 直接对应的部分：交会四边形顶点、四边形直径与加权期望；寻优与等值区域统计略去，完整脚本见支撑材料。

Listing 2: 问题二：期望定位区域直径与最优点、候选区域（Python 3）

```

import math
import numpy as np

X_MAX, ALPHA_DEG = 1500.0, 1.0 #
ALPHA = math.radians(ALPHA_DEG) #

def quad_vertices(x_t, a, b, alpha=ALPHA):
    """S1          x      +-alpha      S2
        S2->S      +-alpha          ABCD          """
    s2 = np.array([a, b])
    psi = math.atan2(-b, x_t - a) # S      S2
    pts = []
    for t1 in (alpha, -alpha):
        u1 = np.array([math.cos(t1), math.sin(t1)])
        for t2 in (psi + alpha, psi - alpha):
            u2 = np.array([math.cos(t2), math.sin(t2)])
            den = u1[0] * u2[1] - u1[1] * u2[0]
            if abs(den) < 1e-14: #
                return None
            s = (s2[0] * u2[1] - s2[1] * u2[0]) / den
            pts.append(s * u1)
    return np.array(pts)

def quad_diameter(x_t, a, b, alpha=ALPHA):
    """          D =          """
    p = quad_vertices(x_t, a, b, alpha)
    if p is None:
        return math.inf
    d = p[:, None, :] - p[None, :, :]
    return float(np.sqrt((d ** 2).sum(-1)).max())

def dbar(a, b, n=800, alpha=ALPHA):
    """          P_T(x_T|S1) = 2 x_T / x_max^2          """
    if abs(b) < 1e-9: #
        return math.inf

```

```

x_t = X_MAX * (np.arange(n) + 0.5) / n
w = 2.0 * x_t / X_MAX ** 2
d = np.array([quad_diameter(float(x), a, b, alpha) for x in x_t])
if not np.isfinite(d).all():
    return math.inf
return float((w * d).sum() / w.sum())

if __name__ == "__main__": #
    best = min(((dbar(x * 1.0, y * 1.0), x, y)
                for x in range(950, 1251, 25) for y in range(500, 901, 25)),
                key=lambda t: t[0])
    print("Dbar_min = %.2f m at (a,b) = (%.0f, %.0f) m" % best)

```

附录 D 问题三、四主程序（待补）

问题三、问题四的策略与算法完成后，其完整可运行源程序将补充于此（对应支撑材料 code/main.py）；该程序负责与模拟器交互，完成多源搜索、定位与清除的全过程。